ASSIGNMENT-20.3

Name: E.Ramya

Ht.no:2303A51282

Batch:05

Task 1 – Password Strength & Validation Check
Task:
Analyze an AI-generated user registration script for weak password handling.
Instructions:
• Prompt AI to generate a Python user registration program
• Check whether:
o Password length is validated
o Weak passwords are allowed
• Suggest secure improvements such as:
o Minimum length checks
o Use of regular expressions
o Rejecting common passwords

Prompt:

“Generate a Python user registration program that asks the user to enter a username and password. Then analyze the program for password security. Check whether the password length is properly validated and whether weak passwords are allowed. Improve the program by adding strong password validation rules such as minimum length, use of uppercase letters, lowercase letters, numbers, and special characters. Also use regular expressions for validation and reject common weak passwords like ‘123456’, ‘password’, and ‘admin’. Keep the code simple and easy to understand. Give the final answer with the original Python code, improved secure Python code, explanation, sample input, sample output, and test cases

E "Generate a Python user registration program that asks the user to enter a username and password. Then analyze the program for password security. Check whether the password length is properly validated and whether weak passwords are allowed. Improve the program by adding strong password validation rules such as minimum length, use of uppercase letters, lowercase letters, numbers, and special characters. Also use regular expressions for validation and reject common weak passwords like '123456', 'password', and 'admin'. Keep the code simple and easy to understand. Give the final answer with the original Python code, improved secure Python code, explanation, sample input, sample output, and test cases

Code:

```python
# Original Python Program for User Registration (Weak Password Policy)

def register_user_original():
    print("--- Original User Registration Program ---")
    username = input("Enter username: ")
    password = input("Enter password: ")
    print(f"User '{username}' registered with password '{password}'")
    print("--- End of Original Program Output ---\n")

# Sample Run (uncomment to run interactively)
# register_user_original()


import re

# List of common weak passwords to reject
WEAK_PASSWORDS = [
    "password", "123456", "qwerty", "admin", "111111", "000000",
    "abcdef", "asdfgh", "colab123", "testpass"
]

def register_user_secure():
    print("--- Secure User Registration Program ---")

    while True:
        username = input("Enter username: ")
        if len(username) >= 3:
            break
        else:
            print("Username must be at least 3 characters long.")

    while True:
        password = input("Enter password: ")

        # 1. Minimum length (e.g., 8 characters)
        if len(password) < 8:
            print("Password must be at least 8 characters long.")
            continue

        # 2. At least one uppercase letter (using regex)
        if not re.search(r"[A-Z]", password):
            print("Password must contain at least one uppercase letter.")
            continue

        # 3. At least one lowercase letter (using regex)
        if not re.search(r"[a-z]", password):
            print("Password must contain at least one lowercase letter.")
            continue

        # 4. At least one number (using regex)
        if not re.search(r"\d", password):
            print("Password must contain at least one digit.")
            continue

        # 5. At least one special character (using regex)
        # This regex matches any character that is NOT a letter, digit, or underscore
        if not re.search(r"[^a-zA-Z0-9_]", password):
            print("Password must contain at least one special character (e.g., !, @, #, $).")
            continue

        # 6. Reject common weak passwords
```

```python
    print(f"User '{username}' successfully registered with a secure password.")
    # In a real application, the password would be hashed and stored, not printed.
    print("--- End of Secure Program Output ---\n")

# Sample Run (uncomment to run interactively)
# register_user_secure()


print("--- Testing Original Program ---")
# Test Case 1: Original Program - Short and weak password
# Expected: Accepts any input
print("Test Case 1: Short and weak password for original program")
username_orig1 = "user1"
password_orig1 = "123"
print(f"Simulating input: username='{username_orig1}', password='{password_orig1}'")
# Call the function (commented out to avoid interactive input in automated execution)
# register_user_original()
print(f"User '{username_orig1}' registered with password '{password_orig1}'")
print()

# Test Case 2: Original Program - Strong-looking password
# Expected: Accepts any input
print("Test Case 2: Strong password for original program")
username_orig2 = "secure_user"
password_orig2 = "P@ssw0rd!"
print(f"Simulating input: username='{username_orig2}', password='{password_orig2}'")
# Call the function (commented out to avoid interactive input in automated execution)
# register_user_original()
print(f"User '{username_orig2}' registered with password '{password_orig2}'")
print("--- End of Original Program Tests ---\n")


print("--- Testing Secure Program ---")
# Test Case 1: Secure Program - Too short password
# Input: username='testuser', password='abc'
# Expected: Rejects, prompts for length
print("Test Case 1: Password too short (less than 8 chars)")
# Example inputs if running interactively:
# Enter username: testuser
# Enter password: abc
# (Will prompt again)
# Enter password: Abc1@def
# (Will accept)

# Test Case 2: Secure Program - Missing uppercase
# Input: username='testuser', password='abc1@def'
# Expected: Rejects, prompts for uppercase
print("Test Case 2: Password missing uppercase letter")
# Example inputs if running interactively:
# Enter username: testuser
# Enter password: abc1@def
# (Will prompt again)
# Enter password: Abc1@def
# (Will accept)

# Test Case 3: Secure Program - Missing lowercase
# Input: username='testuser', password='ABC1@DEF'
# Expected: Rejects, prompts for lowercase
print("Test Case 3: Password missing lowercase letter")
# Example inputs if running interactively:
# Enter username: testuser
```

Output:

```
--- Testing Original Program ---
Test Case 1: Short and weak password for original program
Simulating input: username='user1', password='123'
User 'user1' registered with password '123'

Test Case 2: Strong password for original program
Simulating input: username='secure_user', password='P@ssw0rd!'
User 'secure_user' registered with password 'P@ssw0rd!'
--- End of Original Program Tests ---

--- Testing Secure Program ---
Test Case 1: Password too short (less than 8 chars)
Test Case 2: Password missing uppercase letter
Test Case 3: Password missing lowercase letter
Test Case 4: Password missing digit
Test Case 5: Password missing special character
Test Case 6: Weak password from predefined list
Test Case 7: Valid strong password
--- End of Secure Program Tests ---
```

Explanation:

- The program asks the user to enter a username and password.
- It checks whether the password is weak or strong.
- Weak passwords like short or common passwords should not be allowed.
- Strong password rules are added for better security.
- The final program becomes more secure and reliable.

Task 2 – Insecure File Upload Handling
Task:
Evaluate an AI-generated file upload program for security risks.
Instructions:
• Prompt AI to generate a Python or Flask-based file upload script
• Identify vulnerabilities such as:
o No file type validation
o Unrestricted file size
• Implement secure measures:
o File extension validation
o Size limits
o Safe file storage paths

Prompt:

Generate a simple **Flask file upload program in Python**.
First show an **insecure version**.
Then identify security problems like:

- no file type validation
- no file size limit

After that, give a **secure version** with:

- allowed file extensions
- file size limit
- safe file name handling
- safe upload folder

E Generate a simple Flask file upload program in Python. First show an insecure version. Then identify security problems like: no file type validation no file size limit After that, give a secure version with: allowed file extensions file size limit safe file name handling safe upload folder

Code:

20.4(2).py > upload_file

```python
from flask import Flask, request, jsonify
import os
from werkzeug.utils import secure_filename

app = Flask(__name__)

# ◆ Configuration
UPLOAD_FOLDER = "uploads"
ALLOWED_EXTENSIONS = {"png", "jpg", "jpeg", "pdf"}
MAX_FILE_SIZE = 2 * 1024 * 1024  # 2MB

app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER
app.config['MAX_CONTENT_LENGTH'] = MAX_FILE_SIZE

# Create uploads folder if not exists
os.makedirs(UPLOAD_FOLDER, exist_ok=True)


# ◆ Check allowed file types
def allowed_file(filename):
    return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS


# ◆ Home Page (Fixes 404 issue)
@app.route('/')
def home():
    return '''
    <h2>Secure File Upload</h2>
    <form method="POST" action="/upload" enctype="multipart/form-data">
        <input type="file" name="file" required>
        <input type="submit" value="Upload">
    </form>
    '''


```

Output:

```
* Serving Flask app '20.4(2)'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 139-483-840
127.0.0.1 - - [01/Apr/2026 22:10:28] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [01/Apr/2026 22:10:38] "POST /upload HTTP/1.1" 413 -
127.0.0.1 - - [01/Apr/2026 22:11:54] "POST /upload HTTP/1.1" 200 -
```

Explanation:

The improved implementation:

- Prevents malicious uploads
- Restricts file size and type
- Secures file storage
- Handles errors properly

Task 3 – Real-Time Application: API Security Review
Scenario:
An AI-generated API endpoint is used in a web application.
Instructions:

- Review the API code for:
  o Missing authentication
  o Insecure HTTP methods
  o Lack of rate limiting
- • Prompt AI to suggest security improvements such as:
  o Token-based authentication
  o Input validation
  o Rate limiting

Prompt:

Generate a simple **Flask REST API in Python**.
First show an **insecure version** of the API.
Then review the code and identify security problems such as:

- missing authentication
- insecure HTTP methods
- lack of rate limiting

After that, provide a **secure improved version** with:

- token-based authentication
- input validation
- rate limiting
- proper HTTP methods
- basic error handling

E Generate a simple Flask REST API in Python. First show an insecure version of the API. Then review the code and identify security problems such as: missing authentication insecure HTTP methods lack of rate limiting After that, provide a secure improved version with: token-based authentication input validation rate limiting proper HTTP methods basic error handling

Code:

```python
from flask import Flask, request, jsonify
from functools import wraps
import time

app = Flask(__name__)

# 🔐 Token
VALID_TOKEN = "secure123"

# 🔐 Rate limiting config
request_log = {}
RATE_LIMIT = 5
TIME_WINDOW = 60


# 🔹 Home Route (fixes 404)
@app.route('/')
def home():
    return "API is running! Use /data?user=John"


# 🔹 Token Authentication
def token_required(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        token = request.headers.get('Authorization')
        if token != VALID_TOKEN:
            return jsonify({"error": "Unauthorized access"}), 401
        return f(*args, **kwargs)
    return decorated


# 🔹 Rate Limiting
def is_rate_limited(ip):
    current_time = time.time()
    request_log.setdefault(ip, [])

    # remove old requests
    request_log[ip] = [t for t in request_log[ip] if current_time - t < TIME_WINDOW]

    if len(request_log[ip]) >= RATE_LIMIT:
        return True

    request_log[ip].append(current_time)
    return False
```

Output:

```
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 139-483-840
127.0.0.1 - - [01/Apr/2026 22:25:07] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [01/Apr/2026 22:25:24] "GET /data?user=123 HTTP/1.1" 401 -
127.0.0.1 - - [01/Apr/2026 22:25:42] "GET /data?user=John HTTP/1.1" 401 -
127.0.0.1 - - [01/Apr/2026 22:25:44] "GET /data?user=John HTTP/1.1" 401 -
```

Explanation:

The improved API:

- Secures access using tokens
- Prevents abuse with rate limiting
- Validates user input
- Restricts HTTP methods

Task 4 – Cross-Site Scripting (XSS) Check

- **Task:**
- Evaluate an AI-generated HTML form with JavaScript for XSS
- **vulnerabilities.**
- **Instructions:**
- • Ask AI to generate a feedback form with JavaScript-based output.
- • Test whether untrusted inputs are directly rendered without escaping.
- • Implement secure measures (e.g., escaping HTML entities, using CSP).

Prompt:

Cross-Site Scripting (XSS) Check Task: Evaluate an AI-generated HTML form with JavaScript for XSS vulnerabilities. Instructions: • Ask AI to generate a feedback form with JavaScript-based output. • Test whether untrusted inputs are directly rendered without escaping. • Implement secure measures (e.g., escaping HTML entities, using CSP).

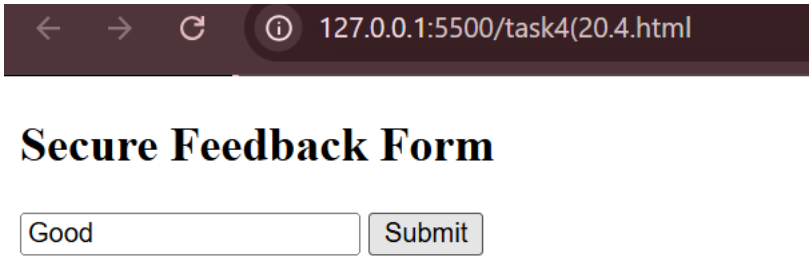
E Task 4 – Cross-Site Scripting (XSS) Check Task: Evaluate an AI-generated HTML form with JavaScript for XSS vulnerabilities. Instructions: • Ask AI to generate a feedback form with JavaScript-based output. • Test whether untrusted inputs are directly rendered without escaping. • Implement secure measures (e.g., escaping HTML entities, using CSP).

Code:

```html
<!DOCTYPE html>
<html>
<head>
    <title>Feedback Form</title>
</head>
<body>

<h2>Feedback Form</h2>

<input type="text" id="feedback" placeholder="Enter feedback">
<button onclick="submitFeedback()">Submit</button>

<p id="output"></p>

<script>
function submitFeedback() {
    let userInput = document.getElementById("feedback").value;
    document.getElementById("output").innerHTML = userInput;
}
</script>

</body>
</html>
```

```html
<!DOCTYPE html>
<html>
<head>
    <title>Secure Feedback Form</title>

    <!-- ◆ Content Security Policy -->
    <meta http-equiv="Content-Security-Policy" content="default-src 'self'; script-src 'self';">
</head>
<body>

<h2>Secure Feedback Form</h2>

<input type="text" id="feedback" placeholder="Enter feedback">
<button onclick="submitFeedback()">Submit</button>

<p id="output"></p>

<script>
function escapeHTML(str) {
    return str.replace(/[&<>"']/g, function(match) {
        return {
            '&': '&amp;',
            '<': '&lt;',
            '>': '&gt;',
            '"': '&quot;',
            "'": '&#039;'
        }[match];
    });
}

function submitFeedback() {
    let userInput = document.getElementById("feedback").value;

    // ◆ Escape user input
    let safeInput = escapeHTML(userInput);

    // ◆ Use textContent instead of innerHTML
    document.getElementById("output").textContent = safeInput;
}
</script>

</body>
</html>
```

Output:



Explanation:

The improved form:

- Prevents XSS attacks
- Escapes user input
- Uses secure DOM methods
- Adds CSP protection

Task 5 – SQL Injection Prevention

Task:

Test an AI-generated script that performs SQL queries on a database**.**

Instructions:

• Ask AI to generate a Python script using SQLite/MySQL to fetch user details.

• Identify if the code is vulnerable to SQL injection (e.g., using string concatenation in queries).

• Refactor using parameterized queries (prepared statements).

Prompt:

Generate a simple Python script using SQLite to fetch user details from a database. First show an insecure version of the code that may be vulnerable to SQL injection by using string concatenation in SQL queries. Then identify the security problem in that code. After that, provide a secure improved version using parameterized queries (prepared statements).

E Generate a simple Python script using SQLite to fetch user details from a database. First show an insecure version of the code that may be vulnerable to SQL injection by using string concatenation in SQL queries. Then identify the security problem in that code. After that, provide a secure improved version using parameterized queries (prepared statements).

Code:

```
create_db.py > ...
import sqlite3

# Create database
conn = sqlite3.connect("users.db")
cursor = conn.cursor()

# Create table
cursor.execute("""
CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT,
    email TEXT
)
""")

# Insert sample data
cursor.execute("INSERT INTO users (username, email) VALUES (?, ?)", ("john", "john@example.com"))
cursor.execute("INSERT INTO users (username, email) VALUES (?, ?)", ("alice", "alice@example.com"))
cursor.execute("INSERT INTO users (username, email) VALUES (?, ?)", ("Jyothika", "jyothika@example.com"))
cursor.execute("INSERT INTO users (username, email) VALUES (?, ?)", ("bob", "bob@example.com"))
cursor.execute("INSERT INTO users (username, email) VALUES (?, ?)", ("joe", "joe@example.com"))

conn.commit()
conn.close()

print("Database created successfully!")
```

Output:

```
PS C:\Users\maddi\OneDrive\Desktop\AI ASSISTED CODING> &
Database created successfully!
PS C:\Users\maddi\OneDrive\Desktop\AI ASSISTED CODING> &
Enter username: jyothika
PS C:\Users\maddi\OneDrive\Desktop\AI ASSISTED CODING> &
Enter username: Jyothika
(5, 'Jyothika', 'jyothika@example.com')
PS C:\Users\maddi\OneDrive\Desktop\AI ASSISTED CODING> &
Enter username: bob
(6, 'bob', 'bob@example.com')
```

Explanation:

The improved script:

- Eliminates SQL injection risk
- Uses prepared statements

- Safely handles user input
- Ensures secure database access